

Algorithmen und Datenstrukturen | AlgDat

Zusammenfassung

INHALTSVERZEICHNIS

1. Suchbäume	2
1.1. Binärer Suchbaum	2
1.1.1. Traversierung	2
1.1.2. Suche	2
1.1.3. Einfügen	2
1.1.4. Löschen	3
1.1.5. Performance	3
1.1.6. Arithmetische Progression	3
2. Sorting	3
3. Text Processing	3
4. Graphs	3

1. SUCHBÄUME

Multimaps: Ungeordnet, `find(k)`, `findAll(k)`, `insert(k,o)`, `remove(e)`

Geordnete Multimaps: Geordnet, `first()`, `last()`, `successors(k)`, `predecessors(k)`

Binäre Suche: Bei einer Multimap, realisiert als array-basierte Sequenz, sortiert nach Key, bei `find(k)`: bei jedem Schritt wird die Anzahl der Kandidaten halbiert, terminiert nach $O(\log n)$ Schritten

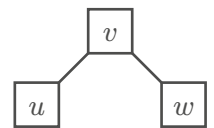
Suchtabelle: Multimap, welche mithilfe einer sortierten Sequenz implementiert wird.

- `find`: $O(\log n)$
- `insert`: $O(n)$
- `remove`: $O(n)$

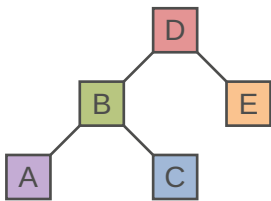
1.1. BINÄRER SUCHBAUM

Ein binärer Baum, welcher Keys (oder Key- Value-Entries) in seinen internen Knoten speichert und folgende Bedingungen erfüllt:

Gegeben sind die drei Knoten u , v , und w . u ist im linken Teilbaum von v und w ist im rechten Teilbaum von v . So gilt: $\text{key}(u) \leq \text{key}(v) \leq \text{key}(w)$



1.1.1. Traversierung



Preorder (W-L-R)
 Postorder (L-R-W)
 Inorder (L-W-R)
 Breadth-First/ Level-Order

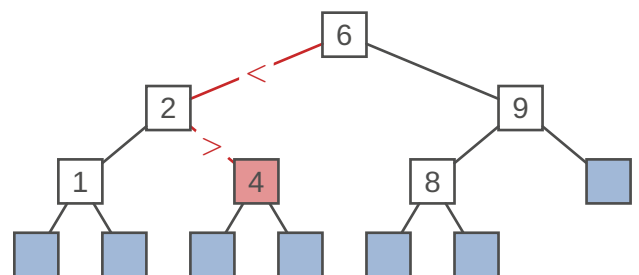
D	B	A	C	E
A	C	B	E	D
A	B	C	D	E
D	B	E	A	C

1.1.2. Suche

```

V search(K k, Node<V> n) {
  if (n.isExternal)
    return null;
  else if (k < n.key)
    return search(k, n.left);
  else if (k > n.key)
    return search(k, n.right);
  else
    return n.value;
}
  
```

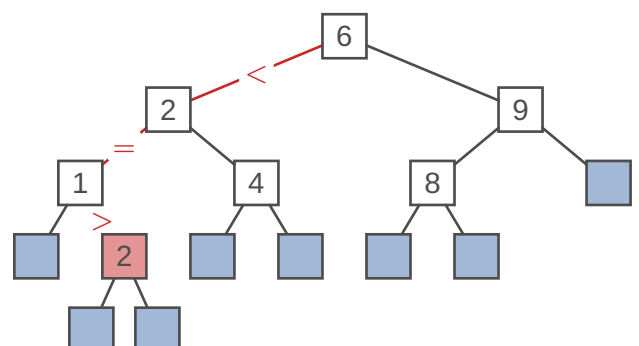
`search(4)`



1.1.3. Einfügen

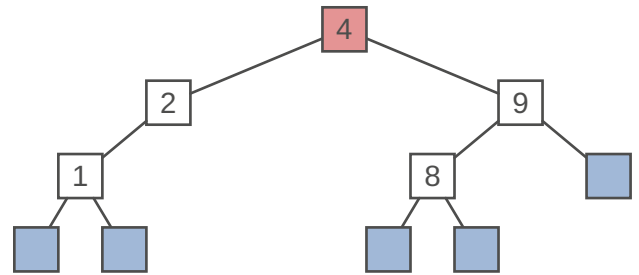
Annahme: der Baum ist eine Multimap und k ist schon im Baum vorhanden: Im jeweils linken Teilbaum von k wird weitergesucht bis man auf ein Blattknoten w stösst

`insert(2)`



1.1.4. Löschen

- finde den internen Knoten w , welcher v in der Inorder-Traversierung folgt
- kopiere $\text{key}(w)$ in den Knoten v
- lösche den Knoten w und sein linkes Kind z (welches ein Blatt sein muss)



1.1.5. Performance

Betrachte eine Map mit n Entries, welcher mit einem binären Suchbaum implementiert ist mit der Höhe h

- nötiger Speicher ist $O(n)$
- Methoden `find`, `insert` und `remove` benötigen $O(h)$
- Die Höhe h ist $O(n)$ im schlechtesten Fall und $O(\log n)$ im besten Fall

1.1.6. Arithmetische Progression

Der Worst-Case tritt z.B. ein, wenn man aufsteigend sortierte Werte in einen binären Such-Baum einfügt. Laut der **Gauss'schen Summenformel** $\frac{n(n+1)}{2}$ ist die Worst-Case Laufzeit somit $O(n^2)$.

TODO:

implementation?

2. SORTING

3. TEXT PROCESSING

4. GRAPHS